

Robot Dynamics Quiz 1

Prof. Marco Hutter, Victor Klemm
Teaching Assistants: Mike Zhang, Clemens Schwarke,
Filip Bjelonic, René Zurbrügg

October 16, 2024

Duration: 1h 15min

Permitted Aids: The exam is open book, which means you can use the script, slides, exercises, etc; The use of internet (besides for licenses) is forbidden; no communication among students during the test is allowed.

1 Instructions

1. Download the ZIP file `RobotDynamics.Quiz1.2024.zip` from Moodle. Extract all contents of this file into a new folder and set MATLAB's¹ current path to this folder.
2. Run `init_workspace` in the MATLAB command line.
3. Run `test_robot_viz` in the MATLAB command line to check that the included robot visualization runs.
4. All problem files that you need to complete are located in the `problems` folder.
5. Run `evaluate_problems` to check if your functions run. This script does not test for correctness. You will get 0 points if a function does not run (e.g., for syntax errors).
6. When the time is up, zip the entire folder and name it `ETHStudentID_StudentName.zip`
Submit this zip-file through Moodle under **Midterm Exam 1 Submission**. You should receive a confirmation email.
7. If the previous step did not succeed, you can email your file to `robotdynamics@leggedrobotics.com`
from your ETH email address with the subject line `[RobotDynamics] ETHStudentID - StudentName`
8. **Important:**
 - (a) Implementations outside the provided templates will not be graded and receive 0 points.
 - (b) The use of functions in MATLAB Toolboxes outside of the base MATLAB installation is not allowed.
 - (c) Helper functions included in the `solutions/pcode` directory are specifically for Questions 4 and 5. Using these functions in the solutions for other questions is prohibited and will result in 0 points.

¹Online version of MATLAB at <https://matlab.mathworks.com/>



Figure 1: Start-up *Zürich Kinematics*' first product: a robot waiter.

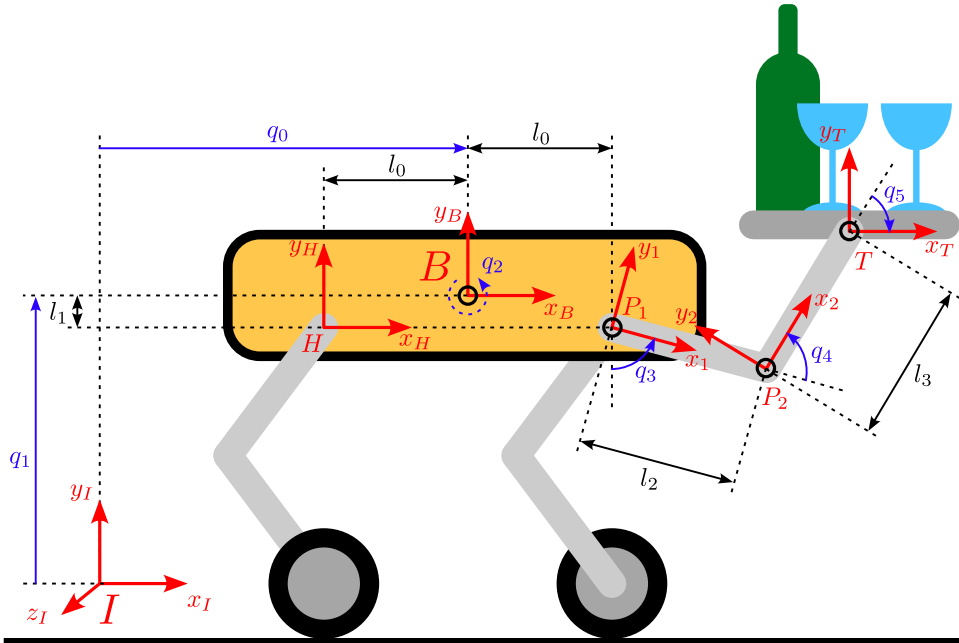


Figure 2: Simplified diagram of the robot waiter. The wheel on one of the front legs was swapped with a serving tray. Thus, the tray is actuated and can rotate around the z_T axis. All four rotational joints, indicated with hollow circles, rotate around the positive z axis (For clarity, the z axis is only drawn for the I frame).

2 Questions

In this quiz, you will model the forward and differential kinematics for the robot waiter shown in Fig. 2. The system has 6 Degrees of Freedom (DoF) in total, 3 for the base and 3 for the leg holding a tray.

The reference frame attached to the base is denoted as $\{B\}$, and is modeled as two prismatic joints q_0 and q_1 , and one revolute joint q_2 . The base moves with respect to the inertial frame $\{I\}$. The reference frames attached to each link of the robot's tray carrying leg are denoted as $\{P_1\}$ and $\{P_2\}$ while the reference frame attached to the tray is denoted as $\{T\}$.

The reference frame $\{H\}$ is centered at the hip joint of the hind leg and rigidly attached to the base of the robot (i.e. the rotation matrix between frames $\{H\}$ and $\{B\}$ is always $\mathbb{I}_{3 \times 3}$).

The generalized coordinates are defined as

$$\mathbf{q} = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \end{bmatrix} \in \mathbb{R}^6. \quad (1)$$

Clarification: The angles q_2, q_3, q_4, q_5 are measured using the right-hand thumb rule. For the state of the scene shown in Fig. 2, q_2 would be zero, q_3 and q_4 would be positive, and q_5 would be negative.

In the following questions, all required parameters are passed to your functions in a structure called **params**. You can access it as follows:

```
1 l0 = params.l0;  
2 l1 = params.l1;  
3 l2 = params.l2;  
4 l3 = params.l3;
```

Question 1.

6 P.

Find the homogeneous transformation between the inertial frame $\{I\}$ and the tray frame $\{T\}$, i.e., the matrix $\mathbf{T}_{IT}$ as a function of the generalized coordinates $\mathbf{q}$.

Hint: You might need to reuse the rotation matrices and translation vectors for question 2.

You should implement your solution in the function `jointToTrayPose.m`

Solution 1.

```

1 function [ T_IT ] = jointToTrayPose_solution(q, params)
2 % Inputs:
3 %     q: a 6x1 vector of generalized coordinates (6, 1).
4 %     params: a struct of parameters.
5 %
6 % Outputs:
7 %     T_IT: Homogenous transform from tray frame {T} to inertial
8 %     frame {I}
9
10 % link lengths (meters)
11 l0 = params.l0;
12 l1 = params.l1;
13 l2 = params.l2;
14 l3 = params.l3;
15
16 % Generalized coordinates
17 q0 = q(1);
18 q1 = q(2);
19 q2 = q(3);
20 q3 = q(4);
21 q4 = q(5);
22 q5 = q(6);
23
24 % Implement your solution here ...
25
26 % Pose of base frame {B} wrt inertial frame {I}
27 r_IB_I = [q0; q1; 0];
28 C_IB = [cos(q2), -sin(q2), 0;
29         sin(q2),  cos(q2), 0;
30         0,        0, 1];
31 T_IB = [ C_IB, r_IB_I;
32         zeros(1,3), 1];
33
34 % Pose of joint 1 frame {P1} wrt base frame {B}
35 r_B1_B = [l0; -l1; 0];
36 C_B1 = [ sin(q3), cos(q3), 0;
37         -cos(q3), sin(q3), 0;
38         0,        0, 1];
39 T_B1 = [ C_B1, r_B1_B;
40         zeros(1,3), 1];
41
42 % Pose of joint 2 frame {P2} wrt joint 1 frame {P1}
43 r_12_1 = [l2; 0; 0];
44 C_12 = [cos(q4), -sin(q4), 0;
45         sin(q4),  cos(q4), 0;
46         0,        0, 1];
47 T_12 = [ C_12, r_12_1;
48         zeros(1,3), 1];
49
50 % Pose of tray frame {T} wrt joint 2 frame {P2}
51 r_2T_2 = [l3; 0; 0];
52 C_2T = [cos(q5), -sin(q5), 0;

```

```

53         sin(q5),  cos(q5),  0;
54         0,         0,  1];
55     T_2T = [      C_2T,  r_2T_2;
56         zeros(1,3),      1];
57
58     % final: Pose of tray frame {T} wrt inertial frame {I}
59     T_IT = T_IB * T_B1 * T_12 * T_2T;
60
61 end

```

Question 2.

4 P.

Compute the position Jacobian ${}_I\mathbf{J}_{IT,P} \in \mathbb{R}^{3 \times 6}$, that fulfills:

$${}_I\mathbf{v}_{IT} = {}_I\mathbf{J}_{IT,P}(\mathbf{q})\dot{\mathbf{q}}, \quad (2)$$

where ${}_I\mathbf{v}_{IT} \in \mathbb{R}^3$ is the linear velocity of point T (the tray frame origin) with respect to a fixed point expressed in frame $\{I\}$.

You should implement your solution in the function `jointToPositionJacobian.m`

Solution 2.

```

1  function [ J_IT_p ] = jointToPositionJacobian_solution(q, params)
2      % Inputs:
3      %   q: a 6x1 vector of generalized coordinates
4      %   params: a struct of parameters
5      %
6      % Outputs:
7      %   J_IT_p: position Jacobian of tray frame {E} wrt inertia ...
8      %           frame {I}
9      %           expressed in inertial frame {I}
10
11     % link lengths (meters)
12     l0 = params.l0;
13     l1 = params.l1;
14     l2 = params.l2;
15     l3 = params.l3;
16
17     % Generalized coordinates
18     q0 = q(1);
19     q1 = q(2);
20     q2 = q(3);
21     q3 = q(4);
22     q4 = q(5);
23     q5 = q(6);
24
25     % Implement your solution here...
26
27     % Rotation of base frame {B} wrt inertial frame {I}
28     C_IB = [cos(q2), -sin(q2), 0;
29             sin(q2),  cos(q2), 0;
30             0,         0,  1];
31
32     % Rotation and translation of joint 1 frame {P1} wrt base frame {B}
33     r_B1_B = [l0; -l1; 0];
34     C_B1 = [ sin(q3), cos(q3), 0;
35             -cos(q3), sin(q3), 0;
36             0,         0,  1];
37
38     % Rotation and translation of joint 2 frame {P2} wrt joint 1 ...
39     % frame {P1}
40     r_12_1 = [l2; 0; 0];
41     C_12 = [cos(q4), -sin(q4), 0;
42            sin(q4),  cos(q4), 0;

```

```

41         0,         0, 1];
42
43 % Translation of tray frame {T} wrt joint 2 frame {P2}
44 r_2T_2 = [l3; 0; 0];
45
46 % Position vector from {P2} to {T} in {I}
47 r_2T_I = C_IB * C_B1 * C_12 * r_2T_2;
48
49 % Position vector from {P1} to {T} in {I}
50 r_12_I = C_IB * C_B1 * r_12_1;
51 r_1T_I = r_12_I + r_2T_I;
52
53 % Position vector from {B} to {T} in {I}
54 r_B1_I = C_IB * r_B1_B;
55 r_BT_I = r_B1_I + r_12_I + r_2T_I;
56
57 % Joint axis vectors in inertial frame {I}
58 nB_I = [0; 0; 1];
59 n1_I = [0; 0; 1];
60 n2_I = [0; 0; 1];
61
62 % Position Jacobian of {T} wrt {I} in {I}
63 J_IT_p = [[1;0;0], [0;1;0], cross(nB_I, r_BT_I), ...
64           cross(n1_I, r_1T_I), cross(n2_I, r_2T_I), zeros(3,1)];
65
66 end

```

Question 3.

2 P.

Compute the rotation Jacobian ${}_I\mathbf{J}_{IT,R} \in \mathbb{R}^{3 \times 6}$, that fulfills:

$${}_I\boldsymbol{\omega}_{IT} = {}_I\mathbf{J}_{IT,R}(\mathbf{q})\dot{\mathbf{q}}, \quad (3)$$

where ${}_I\boldsymbol{\omega}_{IT} \in \mathbb{R}^3$ is the angular velocity of the tray frame $\{T\}$ with respect to the inertial frame $\{I\}$ expressed in frame $\{I\}$.

You should implement your solution in the function `jointToRotationJacobian.m`

Solution 3.

```

1 function [ J_IT_r ] = jointToRotationJacobian_solution(q, params)
2 % Inputs:
3 %     q: a 6x1 vector of generalized coordinates
4 %     params: a struct of parameters
5 %
6 % Outputs:
7 %     J_IT_r: rotation Jacobian of tray frame {E} wrt inertia ...
8 %             frame {I}
9 %             expressed in inertial frame {I}
10
11 % link lengths (meters)
12 l0 = params.l0;
13 l1 = params.l1;
14 l2 = params.l2;
15 l3 = params.l3;
16
17 % Generalized coordinates
18 q0 = q(1);
19 q1 = q(2);
20 q2 = q(3);
21 q3 = q(4);
22 q4 = q(5);
23 q5 = q(6);
24
25 % Implement your solution here...

```

```

25
26 % Joint axis vectors in inertial frame {I}
27 nB_I = [0; 0; 1];
28 n1_I = [0; 0; 1];
29 n2_I = [0; 0; 1];
30 nT_I = [0; 0; 1];
31
32 % Rotation Jacobian of {T} wrt {I} in {I}
33 J_IT_r = [zeros(3,1), zeros(3,1), nB_I, n1_I, n2_I, nT_I];
34
35 end

```

Question 4.

4 P.

You recently launched a startup called *Zürich Kinematics*, specializing in selling your latest flagship robot to restaurants as an automated waiter. The initial deployment of these robots has been successful so far. However, some restaurant owners have recently seen impressive videos of robots performing dance routines on the video portal ClickClack.

They observed that your robots often stand idle while picking up drinks and suggested it would be a great opportunity to entertain customers by incorporating some engaging dance moves during these moments.

Hence, your task is now to implement a tracking controller that is able to track a desired base pose ${}^I\mathbf{p}_{IB,2D}^* = [{}^Ix_{IB}^*, {}^Iy_{IB}^*, {}^I\phi_{IB}^*]^T$ while trying to keep the tray stationary (i.e. ensure zero angular and linear velocity at the tray frame $\{T\}$).

The desired base pose as well as the current generalized coordinates $\mathbf{q}$, are passed as inputs to your MATLAB function.

For this question, we provide the following:

- the 2D base Jacobian ${}^I\mathbf{J}_{IB,2D} \in \mathbb{R}^{3 \times 6}$, that fulfills:

$${}^I\mathbf{w}_{IB,2D} = \begin{bmatrix} {}^I\dot{x}_{IB} \\ {}^I\dot{y}_{IB} \\ {}^I\dot{\phi}_{IB} \end{bmatrix} = {}^I\mathbf{J}_{IB,2D}\dot{\mathbf{q}}. \quad (4)$$

You can call it with `jointToBaseJacobian_solution(q, params);`

- the 2D tray Jacobian ${}^I\mathbf{J}_{IT,2D} \in \mathbb{R}^{3 \times 6}$, that fulfills:

$${}^I\mathbf{w}_{IT,2D} = \begin{bmatrix} {}^I\dot{x}_{IT} \\ {}^I\dot{y}_{IT} \\ {}^I\dot{\phi}_{IT} \end{bmatrix} = {}^I\mathbf{J}_{IT,2D}\dot{\mathbf{q}}. \quad (5)$$

You can call it with `jointToTrayJacobian_solution(q, params);`

- A function to compute the damped pseudo-inverse of a matrix $\mathbf{A}$. You can call it with `pseudoInverseMat_solution(A, lambda)`.

You should implement your solution in the function `kinematicTrajectoryControl.m`.

Your implementation is judged based on how well the robot base tracks a predefined trajectory with minimal linear and angular velocities of the tray. *Do not* modify the tunable parameters of the controller, namely K_p and λ .

A kinematics-simulator is implemented for you in `entertain_control_loop.m`, where the variable `use_solution` is set. You can run this file to visualize the result. If `use_solution` is set to 1, executing this script will show you how the solution should look like. If `use_solution` is set to 0, at each time-step the simulator will use the velocities returned by your own implementation.

Solution 4.

```

1 function [ Dq ] = kinematicTrajectoryControl_solution( q, ...
    p_base_des, params )
2     % Inputs:
3     % q           : current generalized coordinates (6x1)
4     % p_base_des  : desired base pose (3x1)
5     % params      : a struct of parameters
6
7     % Output:
8     % Dq          : generalized coordinates velocity command (6x1)
9
10    % Proportional controller gain
11    K_p = 10;
12
13    % Pseudo-inverse damping coefficient
14    lambda = 1e-2;
15
16    % Generalized coordinates
17    q0 = q(1);
18    q1 = q(2);
19    q2 = q(3);
20    q3 = q(4);
21    q4 = q(5);
22    q5 = q(6);
23
24    % Implement your solution here...
25
26    % Current base pose
27    p_base_curr = [q0; q1; q2];
28    base_err = p_base_des - p_base_curr;
29
30    % Base Jacobian
31    J_base = jointToBaseJacobian_solution(q, params);
32    % Tray Jacobian
33    J_IT = jointToTrayJacobian_solution(q, params);
34
35    J_task = [J_base; J_IT];
36
37    % pseudo-inverse of the analytic Jacobian
38    J_apinv = pseudoInverseMat_solution(J_task, lambda);
39    % desired joint velocities
40
41    % Directly set reference velocity to zero
42    Dq = J_apinv * [K_p * base_err; 0; 0; 0]; % Hint: K_p has the ...
        unit [1/s]
43
44 end

```

Question 5.

2 P.

Today, you find yourself in a tricky situation: Soland Riegwart, a distinguished guest who happens to strongly dislike dogs, has arrived. To make things worse the tray joint at frame $\{T\}$, is malfunctioning and you want to minimize how much it moves unless you enjoy the sound of mechanical failure (which you don't!).

But you, being a sharp ETH student, quickly develop a clever solution:

- You let the robot stand up on its hind legs and convincingly pretend to be a human.
- You minimize any unnecessary movement in the tray joint, hoping to avoid a mechanical fiasco until you can repair it.

Now, because you took Robot Dynamics, you decide to implement a multi-task controller. The tasks are then to stand the robot up, keeping the tray level (no one likes spilled coffee), and to minimize the velocity at the tray joint.

To stand the robot up, it suffices to track a reference pose of the hip joint frame $\{H\}$ denoted by ${}^I\mathbf{p}_{IH,2D}^* = [{}^Ix_{IH}^*, {}^Iy_{IH}^*, {}^I\phi_{IH}^*]^T$

Hint: Can you convert the reference pose of the hip joint frame $\{H\}$ to a reference pose of the base frame $\{B\}$?

The tray is level at the initial state. Therefore, to keep the tray level, it suffices to enforce that the angular velocity of the tray frame $\{T\}$ is zero.

The reference hip joint frame pose as well as the current generalized coordinates $\mathbf{q}$, are passed as inputs to your MATLAB function.

For this question, we provide the following:

- the 2D base Jacobian ${}^I\mathbf{J}_{IB,2D} \in \mathbb{R}^{3 \times 6}$, that fulfills:

$${}^I\mathbf{w}_{IB,2D} = \begin{bmatrix} {}^I\dot{x}_{IB} \\ {}^I\dot{y}_{IB} \\ {}^I\dot{\phi}_{IB} \end{bmatrix} = {}^I\mathbf{J}_{IB,2D}\dot{\mathbf{q}}. \quad (6)$$

You can call it with `jointToBaseJacobian_solution(q, params);`

- the 2D tray Jacobian ${}^I\mathbf{J}_{IT,2D} \in \mathbb{R}^{3 \times 6}$, that fulfills:

$${}^I\mathbf{w}_{IT,2D} = \begin{bmatrix} {}^I\dot{x}_{IT} \\ {}^I\dot{y}_{IT} \\ {}^I\dot{\phi}_{IT} \end{bmatrix} = {}^I\mathbf{J}_{IT,2D}\dot{\mathbf{q}}. \quad (7)$$

You can call it with `jointToTrayJacobian_solution(q, params);`

- A function to compute the damped pseudo-inverse of a matrix $\mathbf{A}$. You can call it with `pseudoInverseMat_solution(A, lambda)`.

You should implement your solution in the function `kinematicStandupControl.m`.

Your implementation is judged based on how well the robot stands up, that the tray stays level, and that movement of the tray joint is minimized. *Do not* modify the tunable parameters of the controller, namely K_p and λ .

A kinematics-simulator is implemented for you in `standup_control_loop.m`, where the variable `use_solution` is set. You can run this file to visualize the result. If `use_solution` is set to 1, executing this script will show you how the solution should look like. If `use_solution` is set to 0, at each time-step the simulator will use the velocities returned by your own implementation.

Solution 5.

```

1 function [ Dq ] = kinematicStandupControl_solution( q, ...
    p_hip_joint_des, params )
2 % Inputs:
3 % q           : current joint angles (6x1)
4 % p_hip_joint_des : desired hip pose in inertial frame (3x1)
5 % params      : a struct of parameters
6
7 % Output:
8 % Dq          : generalized coordinates velocity command (6x1)
9
10 % Proportional controller gain
11 K_p = 1;
12
13 % Pseudo-inverse damping coefficient
14 lambda = 1e-2;
15
16 % Generalized coordinates
17 q0 = q(1);
18 q1 = q(2);
19 q2 = q(3);
20 q3 = q(4);
21 q4 = q(5);
22 q5 = q(6);
23
24 % Implement your solution here...
25
26 % Current base pose
27 p_base_curr = [q0; q1; q2];
28
29 % Transformation to convert hip coordinates to base coordinates
30 R_hip = [cos(q2), -sin(q2), 0;
31          sin(q2),  cos(q2), 0;
32          0,         0, 1];
33
34 p_base_ref = p_hip_joint_des + (R_hip * [params.l0; params.l1; 0]);
35
36 % Calculate error in base frame
37 base_err = p_base_ref - p_base_curr;
38
39 % Jacobian for tracking of base
40 J_base = jointToBaseJacobian_solution(q, params);
41
42 % Tray Jacobian
43 J_IT = jointToTrayJacobian_solution(q, params);
44
45 % Broken joint Jacobian
46 J_broken_joint = [
47     0 0 0 0 0 1
48 ];
49
50 % Full task Jacobian
51 J_task = [
52     J_base;           % Base Pose Tracking
53     J_IT(3,:);        % Tray Angular Velocity
54     J_broken_joint    % Min Broken Joint Velocity
55 ];
56
57 J_task_pinv = pseudoInverseMat_solution(J_task, lambda);
58 w_des = [K_p * base_err; 0; 0]; % Hint: K_p has the unit [1/s]
59 Dq = J_task_pinv * w_des;
60
61 end

```